# **Day 12:** Probabilistic Data Structures & Bloom Filters

Answering approximate questions quickly when exact answers would cost too much memory or latency.

## Why Probabilistic Data Structures Matter

- Exact indexes can become expensive if the dataset is huge
- Probabilistic structures trade a small amount of accuracy for major memory savings.
- Useful if a fast "probably yes" or "definitely no" answer is enough

## Bloom Filter Basics

- A **Bloom filter** is a bit array plus multiple Hash functions
- Adding an item sets several bit positions
- Checking an item returns "definitely not present" or "might be present"

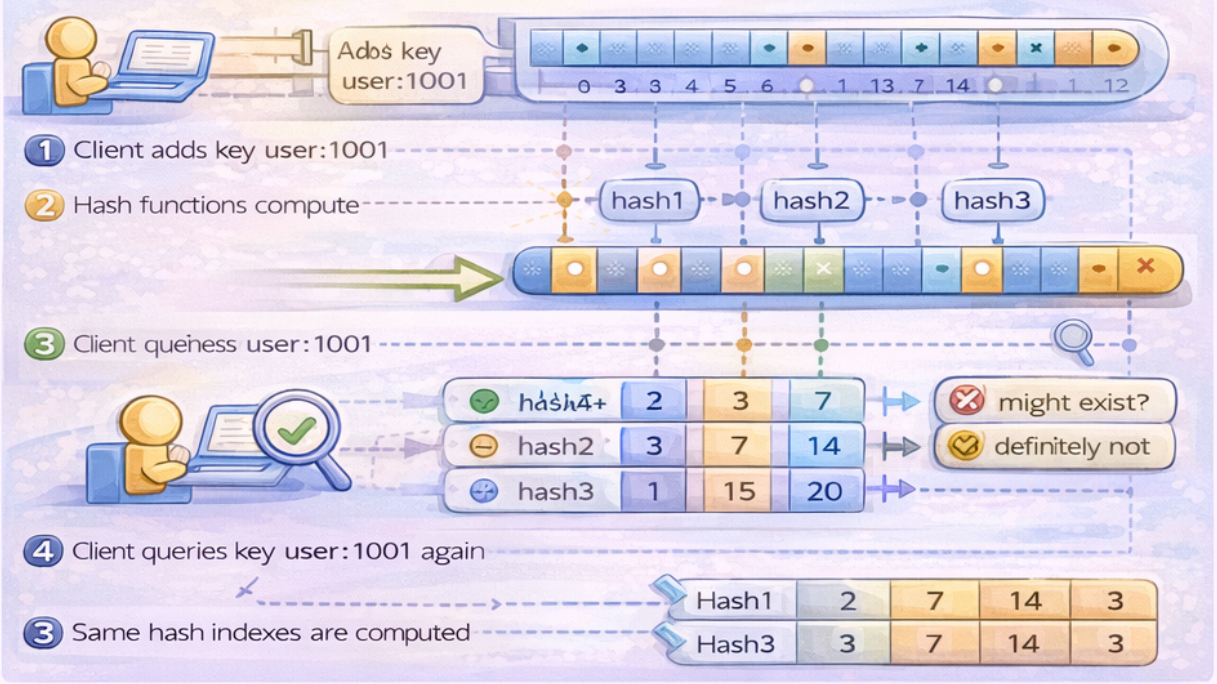
## False Positives & No False Negatives

- May say an item might exist even when it does not
- Standard Bloom filter should produce no false negatives
- More items inserted → higher false-positive rate

## Where Bloom Filters Help

- Prevent unnecessary database reads for missing keys
- Filter duplicate events before downstream processing
- Reduce disk lookups in storage engines

## Sample Bloom Filter Data Flow




Bloom filters make sense when "definitely not present" is more valuable than exact matches, especially at large scale.

# Overview from README.md

# Day 12: Probabilistic Data Structures and Bloom Filters

Day 12 focuses on how large systems answer approximate questions quickly when exact answers would cost too much mem

It covers:
- probabilistic data structures
- approximate membership checks
- Bloom filters and false positives
- memory vs accuracy trade-offs
- cache penetration and duplicate suppression
- cardinality estimation and sketch-based thinking

In short, Day 12 is about using approximation deliberately so systems can stay fast and cheap at scale.

![Day 12](./Day12.png)

[Day 12 PDF](./System_Design_Day_12.pdf)

## Core ideas

### Why probabilistic data structures matter
- Exact indexes can become expensive when the dataset is huge.
- Probabilistic structures trade a small amount of accuracy for major memory savings.
- They are useful when a fast "probably yes" or "definitely no" answer is enough.

### Bloom filter basics
- A `Bloom filter` is a bit array plus multiple hash functions.
- Adding an item sets several bit positions.
- Checking an item returns `definitely not present` if any required bit is unset, or `might be present` if all bits are set.

### False positives and no false negatives
- A Bloom filter can say an item `might exist` even when it does not.
- A correctly implemented standard Bloom filter should not produce false negatives for added items.
- As more items are inserted, the false-positive rate increases.

### Where Bloom filters help
- Prevent unnecessary database reads for definitely missing keys
- Filter duplicate events before expensive downstream work
- Reduce disk lookups in storage engines
- Pre-check membership in distributed caches or search systems

### Trade-offs
- Bloom filters do not store the original values.
- Standard Bloom filters do not support exact deletion.
- You need to size the bit array and hash count based on expected volume and acceptable false-positive rate.

## Day-12 sample: Bloom Filter

This repository includes a small Bloom filter sample in both Python and Java.

# File: bloom_filter.py

```
from __future__ import annotations

import hashlib
import math


class BloomFilter:
    def __init__(self, size_bits: int, hash_count: int) -> None:
        if size_bits <= 0:
            raise ValueError("size_bits must be positive")
        if hash_count <= 0:
            raise ValueError("hash_count must be positive")

        self.size_bits = size_bits
        self.hash_count = hash_count
        self._bits = bytearray((size_bits + 7) // 8)
        self._insertions = 0

    def add(self, item: str) -> None:
        for index in self._indexes(item):
            self._set_bit(index)
        self._insertions += 1

    def might_contain(self, item: str) -> bool:
        return all(self._get_bit(index) for index in self._indexes(item))

    def fill_ratio(self) -> float:
        bits_set = sum(byte.bit_count() for byte in self._bits)
        return bits_set / self.size_bits

    def estimated_false_positive_rate(self) -> float:
        m = self.size_bits
        k = self.hash_count
        n = self._insertions
        return (1 - math.exp((-k * n) / m)) ** k

    def _indexes(self, item: str) -> list[int]:
        if not item:
            raise ValueError("item must be non-empty")

        return [self._index_for_seed(item, seed) for seed in range(self.hash_count)]

    def _index_for_seed(self, item: str, seed: int) -> int:
        digest = hashlib.sha256(f"{seed}:{item}".encode("utf-8")).digest()
        return int.from_bytes(digest[:8], byteorder="big") % self.size_bits

    def _set_bit(self, index: int) -> None:
        byte_index, bit_offset = divmod(index, 8)
        self._bits[byte_index] |= 1 << bit_offset

    def _get_bit(self, index: int) -> bool:
        byte_index, bit_offset = divmod(index, 8)
        return bool(self._bits[byte_index] & (1 << bit_offset))
```

# File: demo.py

```python
from bloom_filter import BloomFilter


def print_check(filter_: BloomFilter, item: str) -> None:
    verdict = "might be present" if filter_.might_contain(item) else "definitely not present"
    print(f"{item:>12} -> {verdict}")


def main() -> None:
    bloom = BloomFilter(size_bits=128, hash_count=3)

    inserted_items = [
        "user:1001",
        "user:1002",
        "user:1003",
        "order:9001",
        "order:9002",
    ]

    print("adding items")
    for item in inserted_items:
        bloom.add(item)
        print(f"  added {item}")

    print("\nmembership checks")
    for item in ["user:1001", "user:5000", "order:9002", "order:9999"]:
        print_check(bloom, item)

    print("\nfilter stats")
    print(f"  fill ratio: {bloom.fill_ratio():.2%}")
    print(f"  estimated false positive rate: {bloom.estimated_false_positive_rate():.4%}")


if __name__ == "__main__":
    main()
```

# File: BloomFilter.java

```java
import java.nio.ByteBuffer;
import java.nio.charset.StandardCharsets;
import java.security.MessageDigest;
import java.security.NoSuchAlgorithmException;
import java.util.BitSet;

public class BloomFilter {
    private final int sizeBits;
    private final int hashCount;
    private final BitSet bits;
    private int insertions;

    public BloomFilter(int sizeBits, int hashCount) {
        if (sizeBits <= 0) {
            throw new IllegalArgumentException("sizeBits must be positive");
        }
        if (hashCount <= 0) {
            throw new IllegalArgumentException("hashCount must be positive");
        }

        this.sizeBits = sizeBits;
        this.hashCount = hashCount;
        this.bits = new BitSet(sizeBits);
        this.insertions = 0;
    }

    public void add(String item) {
        for (int index : indexes(item)) {
            bits.set(index);
        }
        insertions += 1;
    }

    public boolean mightContain(String item) {
        for (int index : indexes(item)) {
            if (!bits.get(index)) {
                return false;
            }
        }
        return true;
    }

    public double fillRatio() {
        return (double) bits.cardinality() / sizeBits;
    }

    public double estimatedFalsePositiveRate() {
        return Math.pow(1 - Math.exp((-1.0 * hashCount * insertions) / sizeBits), hashCount);
    }

    private int[] indexes(String item) {
        if (item == null || item.isBlank()) {
            throw new IllegalArgumentException("item must be non-empty");
        }

        int[] indexes = new int[hashCount];
        for (int seed = 0; seed < hashCount; seed += 1) {
            indexes[seed] = indexForSeed(item, seed);
        }
        return indexes;
    }

    private int indexForSeed(String item, int seed) {
        try {
            MessageDigest digest = MessageDigest.getInstance("SHA-256");
            byte[] hashed = digest.digest((seed + ":" + item).getBytes(StandardCharsets.UTF_8));
            long value = ByteBuffer.wrap(hashed, 0, Long.BYTES).getLong();
            return (int) Math.floorMod(value, sizeBits);
        } catch (NoSuchAlgorithmException e) {
            throw new IllegalStateException("SHA-256 algorithm unavailable", e);
        }
    }
}
```

# File: Main.java

```
public class Main {
    public static void main(String[] args) {
        BloomFilter bloom = new BloomFilter(128, 3);

        String[] insertedItems = {
            "user:1001",
            "user:1002",
            "user:1003",
            "order:9001",
            "order:9002"
        };

        System.out.println("adding items");
        for (String item : insertedItems) {
            bloom.add(item);
            System.out.println("  added " + item);
        }

        System.out.println("\nmembership checks");
        printCheck(bloom, "user:1001");
        printCheck(bloom, "user:5000");
        printCheck(bloom, "order:9002");
        printCheck(bloom, "order:9999");

        System.out.println("\nfilter stats");
        System.out.printf("  fill ratio: %.2f%%%n", bloom.fillRatio() * 100);
        System.out.printf(
            "  estimated false positive rate: %.4f%%%n",
            bloom.estimatedFalsePositiveRate() * 100
        );
    }

    private static void printCheck(BloomFilter bloom, String item) {
        String verdict = bloom.mightContain(item) ? "might be present" : "definitely not present";
        System.out.printf("%12s -> %s%n", item, verdict);
    }
}
```